→ then load .env file in your Python script :-

```
from dotenv import load_dotenv
import os

load_dotenv()
print(os.getenv("my_password"))
print(os.getenv("API_KEY"))
```

then run python script

```
python app.py
```

```
» abcd
» 1234abcd
```

Python Operators :

Operators are used to perform operations on variables & values.

1) Arithmatic operator :-

+ → addition 10 + 5 → 15
- → subtraction 10 − 5 → 5
* → multiplication 10 * 5 → 50
/ → Division (gives result in point) 10/3 → 3.333
// → Floor division (gives integer result) 10//3 → 3
% → modulus (gives remainder) 10 % 3 → 1
** → Exponentiation (power) 2 ** 3 → 8

2) Assignment operators :-

=	a = 5
+=	a = a+5 or a += 5
-=	a = a−2 or a -= 2
*=	a = a * 2 or a *= 2
/=	a = a/2 or a /= 2

`|=`	`a = a | 2` or	`a |= 2`
`%=`	`a = a % 2` or	`a %= 2`
`**=`	`a = a ** 2` or	`a **= 2`

3) Comparison (Relational) operators :-

used to compare two values & result is either True or False.

			output
`==`	Equal to	`5 == 5`	True
`!=`	Not equal to	`5 != 3`	True
`>`	Greater than	`5 > 3`	True
`<`	Less than	`5 < 3`	False
`>=`	greater than equals to	`5 >= 3`	~~False~~ True
`<=`	less than equal to	`5 <= 3`	False

4) Logical operators

used to perform conditional statements

→ and — True if both are true

e.g: `print ((5>3) and (8>6))`
>> True

→ or — True if any one is true

e.g: `print ((5>3) or (8<6))`
>> True

→ not — Inverts result

`print(not (5>3))` >> False

5) Identity operators :-

used to compare memory locations (object identity)

→ is — True if both refer to same object — a is b

→ is not — True if not same object — a is not b

e.g.-

```
x = ["apple", "banana"]
y = ["apple", "banana"]
z = x
print (x is z)      >> True
print (x is y)      >> False
print (x == y)      >> True (same values)
print (x is not y)  >> True
```

6) Membership operator :-

→ used to test if value exist in a sequence (string, list) etc.

→ in — returns true if found

→ not in — returns true if not found

e.g.-

```
fruits = ["apple", "banana", "cherry"]
print ("apple" in fruits)      >> True
print ("grape" not in fruits)  >> True
```

Operators Precedence :-

Highest to lowest

`()` → Parenthesis

`**` → Exponentiation

`+x, -x, ~x` → Unary plus, Unary minus, & bitwise not.

`*, /, //, %` → multiplication, Division, floor division & modulus.

`+, -` → Addition & Subtraction

`<< >>` → Bitwise left & right shifts

`&` → Bitwise AND

`^` → Bitwise XOR

`|` → Bitwise OR

`==, !=, >, >=, <, <=, is, is not, in, not in` → comparison, Identity & Membership operators.

`not` → Logical NOT

`and` → " AND

`or` → " OR

Conditional statement

Basic syntax:-

```
if condition:
    # code to execute if condition is true
elif another condition:
    # code to execute if above is false but this
      is true
else:
    # code to be executed if above conditions are
      false.
```

e.g:-
```
a = 10
if a > 5:
    print("a is greater than 5")
```

Output:-

a is greater than 5

e.g.2 if - else

```
x = 3
if x % 2 == 0:
    print ("even number")
else:
    print("odd number")    # output
```

e.g.3 if - elif - else:-

```
marks = 85
if marks >= 90:
    print("grade : A")
elif marks >= 75:
    print ("grade: B")    # output
elif marks >= 60:
    print ("grade : C")
else:
    print ("grade : D")
```

Shorthands:

for if:-

eg. a = 5
b = 2
if a > b : print ("a is greater than b")

for if - else:-

a = 2
b = 330
print ("A") if a > b else print ("B")

e.g.4 Nested if statements

```
x = 15
if x >= 10:
    if x < 20:
        print ("x is between 10 & 20")  # output
```

e.g.5 Using logical operator

```
age = 25
if age > 18 and age < 30:
    print ("Eligible for program")  # output
```

Nested if-else. another example:

```
age = 25
has_license = True
if age >= 18 :
    if has_licence :
        print ("You can drive")
    else:
        print ("You need a licence")
else:
    print ("You are too young to drive")
```

Lists :-

A list is a collection of items that can store multiple values in a single variable

Lists are:

→ ordered
→ mutable (changeable) → we can add, remove items in lists after it is been created.
→ Allow duplicates
→ can store diff. data types (integers, strings, floats etc)

Creating list :-

We can create list using square brackets [ ]

```
fruits = ["Apple", "orange", "cherry"]
print (fruits)
```

Output

```
['apple', 'banana', 'cherry']
```

∴ list with multiple data types.

```
mixed_data_type = ["Alok", 26, 2.4, True]
```

→ `print (len (fruits))` # this will print the no. of items in the list

```
>> 3
```

→ `print (len (fruits[0]))` # this will print the length of first index element or first item of the list

```
>> 5
```

The list() constructor :-

use list contructor to make a list

```
thelist = list (("apple", "bananan", "mango"))
print (thelist)
```

Output

```
['apple', 'bananan', 'mango']
```

There are 4 collection data types in python :

1) Lists : Ordered, changeable, Allow duplicate
2) Tuple : Ordered, Unchangeable, Allow duplicate
3) Set : Unordered, unchangeable, unindexed, No duplicate
4) Dictionary : Ordered, changeable, No duplicacy.

Accessing list items :-

```
fruits = ["apple", "mango", "cherry"]
print (fruits [0])   # apple
print (fruits [1])   # mango.
print (fruits [-1])  # cherry (negative indexing
```

Slicing lists :-

```
cars = ["bmw", "mercedes", "audi", "ferrari", "lambo"]
→ print(cars [1:4])   # prints elements from index
                        1 to 3
>> ['mercedes', 'audi', 'ferrari']

→ print (cars [:3])   # prints from index 0 to 2
>> ['bmw', 'mercedes', 'audi']

→ print (cars [::-1])   # prints list in reverse order
>> ['lambo', 'ferrari', 'audi', 'mercedes', 'bmw']

→ print (cars [-4:-1])
>> ['mercedes', 'audi', 'ferrari']
```

Checking if item exist in list:-

```
if "bmw" in cars:
    print("most good looking car")  # output
```

Change list items:

1) change specific item value

```
cars[1] = "Bentley"
print(cars)
```

» ['bmw', 'Bentley', 'audi', 'ferrari', 'lambo']

2) change a range of item values:

```
cars[0:3] = ["cadillac", "supra", "maybach"]
print(cars)
```

» ["cadillac", "supra", "maybach", 'ferrari', 'lambo']

Insert new items:

To insert new list item, without replacing any of the existing values, we can use the insert() method.

→ insert() method, inserts an item at the specified index.

```
cars.insert(0, "BMW")
print(cars)
```

(0 → index no; "BMW" → item value to be inserted)

» ['Bmw', 'cadillac', 'supra', 'maybach', 'ferrari', 'lambo']

Append item :-

To add an item at the end of the list, use

append()

```
→ cars.append("Aston Martin")
  print(cars)
```

```
>> ['BMW', 'cadillac', 'supra', 'maybach', 'ferrari',
    'lambo', 'Aston Martin']
```

Extend list

To append elements from another list to the current list, use extend() method

```
→ ind_cars = ["maruti", "mahindra", "Thar"]
  cars.extend(ind_cars)
  print(cars)        # these elements will be added to
                       the end of the list.
```

```
>> ['BMW', 'cadillac', 'supra', 'maybach', 'ferrari',
    'lambo', 'Aston Martin', 'maruti', 'mahindra', 'Thar']
```

Imp. Note :-

The extend method does not have to append lists, you can add any iterable object (tuples, sets, dictionaries)

```
→ more_cars = ("Toyota", "Defender", "Range Rover")
              # This is a tuple

  cars.extend(more_cars)
  print(cars)
```

```
>> ['BMW', 'cadillac', 'supra', 'maybach', 'ferrari', 'lambo',
  'Aston Martin', 'maruti', 'mahindra', 'Thar', 'Toyota', 'Defender',
    'Range Rover']
```

Remove list items :-

remove method removes first occurrence

1) Remove specified item :-

```
cars.remove ("Range Rover")
print (cars)   # prints cars list after removing
                that particular item.
```

2) Remove item at specified index :

the pop method removes the specified index.

→ cars.pop (1) → This will remove the second item
print (cars) from the list

∴ If we do not specify the index, pop() method removes the last item.

Del() keyword also removes the specified index:

→ del cars [1] # removes the element of the index no
print (cars) 1 or simply removes second elem.

del keyword can also delete the list completely :-

del cars # this will delete the entire list.

3) clear the list :

→ clear() method empties the list.
→ the list still remains, but it has no contents

→ cars.clear()
→ print (cars)

output

[] # empty list

Loop through a list :-

You can loop through the list items by using a for loop.

e.g:-

```
fruits = ["apple", "banana", "mango"]
for x in fruits:
    print(x)
```

Output

apple
banana
mango

Loop through index numbers :-

```
for i in range(len(fruits))
    print(fruits[i])
```

Sorting in list

List objects have sort() method that will sort the list alphanumerically.

e.g:-

```
games = ["PUBG", "COD", "BGMI", "COC"]
games.sort()
print(games)
```

Output

```
['BGMI', 'COC', 'COD', 'PUBG']
```

For numericals :-

```
num = [10, 8, 50, 1, 40]
num.sort
print(num)  # output --> [1, 8, 10, 40, 50]
```

∴ By default sort method is case sensitive, resulting all capital letters being sorted before lower case letters.

Reverse order:-

→ reverse() method reverses the current sorting order of the elements.

eg.

```
list = ["banana", "Orange", "Kiwi", "cherry"]
list.reverse()
print(list)
```

Output:

['cherry', 'Kiwi', 'Orange', 'banana']

Copy List:

```
mylist = list.copy()
print(mylist)
```

Output

Nested List:

A list inside another list.

e.g.

```
matrix = [[1, 2, 3], [4, 5, 6]]
print(matrix[1][2])
```

Output

6

Tuples :-

→ Tuple is just like list - it can store multiple items in a single variable - but main diff. is

→ Tuples are immutable (immutable)
means we can not change, add or remove items once the tuple is created.

→ ordered & allow duplicate values.

→ Tuples are written inside parenthesis.

e.g:- fruits = ("Apple", "mango", "Banana")
print (fruits)

output

('Apple', 'mango', 'Banana')

One item tuple :-

thistuple = ("apple",) → we have include comma
print (type (thistuple)) ---> this is a tuple with one item

tuple = ("apple")
print (type (tuple)) ---> this returns string

A tuple can contain diff. data types :-

tuple 1 = ("abc", 34, True, 4.0)

Accesing tuple :-

print (fruits [0]) # output --> Apple.
print (fruits [-1]) # Banana.

```
print (fruits [0:3])    # range of indexing or slicing.
```

Output

('Apple', 'Cherry', 'Banana')

Check if item exists :-

```
if "Apple" in fruits:
    print ("Yes 'Apple' exists in fruits tuple")
```

Change tuple values :-

as tuples are immutable, we cannot change its values. But there is a workaround. We can convert the tuple into a list, change the list & convert the list back into a tuple.

e.g.:-

```
x = ("alok", "abhay", "shivam", "Abhishek")
y = list (x)    # this converts x tuple into list and
                  store list in y variable.
y[1] = "Akshat" --> modify list item
x = tuple(y)    --> convert y into tuple
print (x)
```

Output

('alok', 'Akshat', 'shivam', "Abishek")

We can also add items :-

```
y.append ("Dev")
x = tuple (y)
print(x)
```

Output

("alok', 'Akshat', 'shivam', 'abhishek', 'Dev')

Delete a tuple:

del tuple_name → name of the tuple you want to delete.

del keyword deletes the tuple completely.

Looping through a tuple :-

```
name = ("alok", "dev", "pankaj")
for x in name:
    print(x)
```

output

alok
dev
pankaj

Joining Tuple:-

```
tuple1 = (1,2,3)
tuple2 = (4,5,6)
result = tuple1 + tuple2
print(result)
```

Output:-

(1, 2, 3, 4, 5, 6)

Nested tuples :-

```
nested = ((1,2,3), (4,5), (6,7))
print(nested[1][0])
```

output:

4

Tuple Methods:

1) count():

returns the no. of times a specified value occur in a tuple.

```
names = ("Alok", "Abhay", "Alok", "Akshat", "Alok")
print(names.count("Alok"))
```

Output:

3

2) index()

returns the index (position) of the first occurrence of a specified value.

→ `print(names.index("Abhay"))`

Output:-

1

Why use tuples instead of lists?

→ tuples are faster

→ tuples are safe (data cannot be changed accidently)

→ tuples can be used as keys in dictionaries, lists cannot.

Loops :

loop is used to repeat a block of code multiple times.

There are two types of loops in python:-

→ for loop.
→ while loop.

For loop:-

The for loop is used to iterate over a sequence (like a list, tuple, string or range)

Suppose we want to print numbers 1 to 100, without we have to write it like print(1), print(2) - - - print(100) manually, but with loops :-

```
for i in range(1, 101):
    print(i)
```

output:

```
1
2
.
.
.
100
```

```
for i in range(5):
    print(i)
```

output

```
0
1
2
3
4
```

Loop through list:-

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
    print(x)
```

output:-

```
apple
banana
cherry
```

Loop through string:-

```
name = "Alok"
for i in name:
    print(i)
```

output:

```
A
l
o
k
```

Loop with range (start, stop, step)

→ not included

```
for i in range(2, 10, 2):
    print(i)
```

output

```
2
4
6
8
```

Break statement :-

with break statement we can stop the loop before it has looped through all the items.

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
    print(x)
    if x == "banana"
    . break.
```

Output :

```
apple
banana
```

```
names = ["alok", "abhishek", "shivam", "Akshat]
for x in names[1]:
    print(x)
```

Output :

```
a
b
h
i
s
h
e
k
```

while loop :-

while loop keeps running as long as the condition is True.

eg.

```
i = 1
while i <=5;
    print i
    i +=1
```

Output:

```
1
2
3
4
5
```

Infinite loop

```
while True:
    print("Infinite")
```

while loop with Break :-

```
i = 1
while i <=10:
    if i == 6:
        break
    print(i)
    i += 1
```

Output :

```
1
2
3
4
5
```

Continue Statement :-

```
i = 0
while i < 6:
    i += 1
    if i == 3: 
        continue
    print(i)
```

skips this and goes to next execution

∴ that's why 3 is not printed

output

1
2
4
5
6

→ Continue statement is used to skip the current iteration of the loop and proceed to next one.

→ It can be used in both "for" & "while" loops, enabling you to bypass certain iterations based on a condition.

Taking Input from user :

```
numbers = input("Enter a number: ")
print(numbers)
```

output

Enter a number: 6 → this is the no. user provided as input

5 → prints the number.

Exceptional Handling :-

An exception is an error that occur during program execution which stops the normal flow of a program.

e.g. of exceptions :-

ZeroDivisionError → dividing by zero
ValueError → invalid type conversion.
FileNotFoundError → trying to open a file that doesn't exist.
IndexError → accessing an invalid list index

e.g. :-

```
a = int(input("Enter a number: "))
b = int(input("Enter another number: "))
print(a/b)
```

In this code if user enters b = 0, this will throw an error named ZeroDivisionError.

- So to handle this we can use exceptional handling

```
try:
    a = int(input("Enter a number: "))
    b = int(input("Enter another number: "))
    print(a/b)
except ZeroDivisionError:        → type of error you want to handle
    print("You cannot divide by zero")
    continue # use this to continue further
```

Output

```
Enter a number: 2
Enter another number: 0
You cannot divide by zero
```

} Here error is handled and no error shown except error message we write.

```
try:
  # code
except name of error:
  #code
except another name of error:
  # code
```

Else :

We can use else keyword to define a block of code to be executed if no errors were raised.

e.g.-

```
try:
  print("Hello")
except:
  print("something went wrong")
else:
  print("Everything is fine")
```

Output

```
Hello
Everything is fine
```

Output :

Finally :

finally runs always — whether an error occur or not.
→ used to close files, release resources or clean up

e.g.-

```
try:
  f = open("myfile.txt")
  print(f.read())
except FileNotFoundError:
  print("File not found!")
finally:
  print("Execution complete (closing file).")
```

Output

File not found! → output of except block
Execution complete (closing file). → output of finally

Another example:-

Try to open and write to a file that is not writable.

```
try:
    f = open("demofile.txt")
    try:
        f.write("Lorum Ipsum")
    except:
        print("Something went wrong when writing!")
    finally:
        f.close()
except:
    print("Something went wrong while opening")
```

Raise an exception:-

You can create errors manually using raise keyword.

e.g.

```
x = -5
if x < 0:
    raise ValueError("Negative numbers not allowed")
```

Output

ValueError: " Negative numbers not allowed

Final example using all these:

```
try:
    num1 = int(input("Enter a number: "))
    num2 = int(input("Enter another number: "))
    print(num1 / num2)
except ZeroDivisionError:
    print("Cannot divide by zero")
except ValueError:
    print("Please enter a valid input")
else:
    print("Program executed")
finally:
    print("Program finished")
```

Outputs:

Enter a number: 2
Enter another number: 5
0.4
Program executed
Program finished

Enter a number: Alok
Please enter a valid input
Program finished

Enter a number: 2
Enter another number: 0
Cannot divide by zero
Program finished.